Title : Analyzing Weather Data from OpenWeatherMap API

In [2]:
```
import requests
import pandas as pd
import datetime
```

In [3]:
```
# Set your OpenWeatherMap API key
api_key = 'fb365aa6104829b44455572365ff3b4e'
```

In [4]:
```
# Set the location for which you want to retrieve weather data
lat = 18.184135
lon = 74.610764
```

In [5]:
```
# https://openweathermap.org/api/one-call-3
# how   How to use api call
# Construct the API URL
api_url = f"http://api.openweathermap.org/data/2.5/forecast?lat={lat}&lon={lon}&appid={api_ke
```

In [8]:
```
# Send a GET request to the API
response = requests.get(api_url)
weather_data = response.json()
weather_data.keys()
len(weather_data['list'])
weather_data['list'][0]['weather'][0]['description']
```

Out[8]: 'scattered clouds'

In [11]:
```
# Getting the data from dictionary and taking into one variable
# Extract relevant weather attributes using list comprehension
temperatures = [item['main']['temp'] for item in weather_data['list']]

# It will extract all values (40) and putting into one variable
timestamps = [pd.to_datetime(item['dt'], unit='s') for item in weather_data['list']]
temperature = [item['main']['temp'] for item in weather_data['list']]
humidity = [item['main']['humidity'] for item in weather_data['list']]
wind_speed = [item['wind']['speed'] for item in weather_data['list']]
weather_description = [item['weather'][0]['description'] for item in weather_data['list']]
```

In [21]:
```
# Create a pandas DataFrame with the extracted weather data
weather_df = pd.DataFrame({'Timestamp': timestamps,
                           'Temperature': temperatures,
                           'humidity': humidity,
                           'wind_speed':wind_speed,
                           'weather_description': weather_description})
```

In [22]:
```
# Set the Timestamp column as the DataFrame's index
weather_df.set_index('Timestamp', inplace=True)
max_temp = weather_df['Temperature'].max()
print(f"Maximum Temperature - {max_temp}")
min_temp = weather_df['Temperature'].min()
print(f"Minimum Temperature - {min_temp}")
```

```
Maximum Temperature - 305.27
Minimum Temperature - 292.37
```

In [23]:
```
# Clean and preprocess the data # Handling missing values
weather_df.fillna(0, inplace=True) # Replace missing values with 0 or appropriate value
```

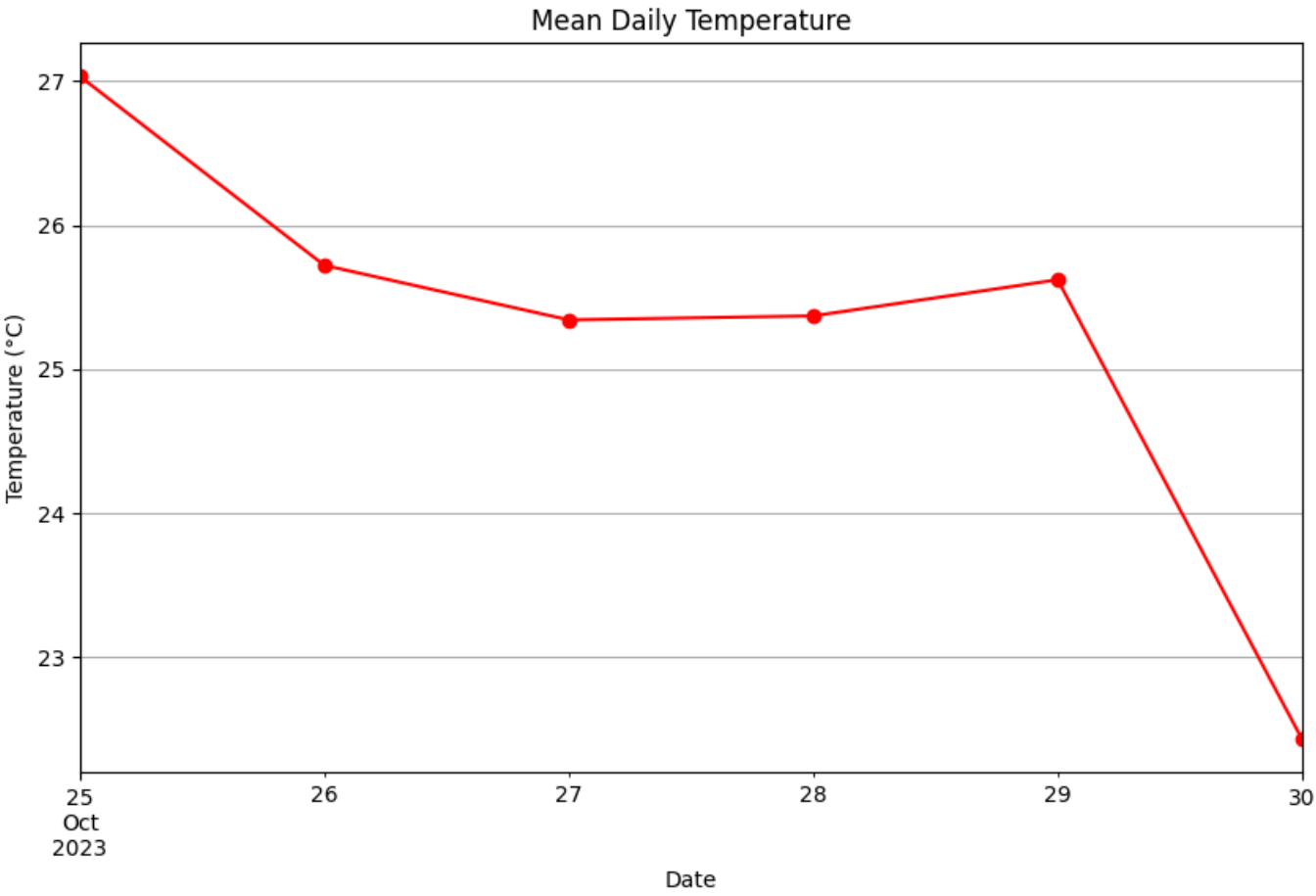
In [24]:
```
# Handling inconsistent format (if applicable)
weather_df['Temperature'] = weather_df['Temperature'].apply(lambda x: x - 273.15 if isinstanc
```

In [25]:
```
# Convert temperature from Kelvin to Celsius
# Print the cleaned and preprocessed data print(weather_df)
print(weather_df)
```

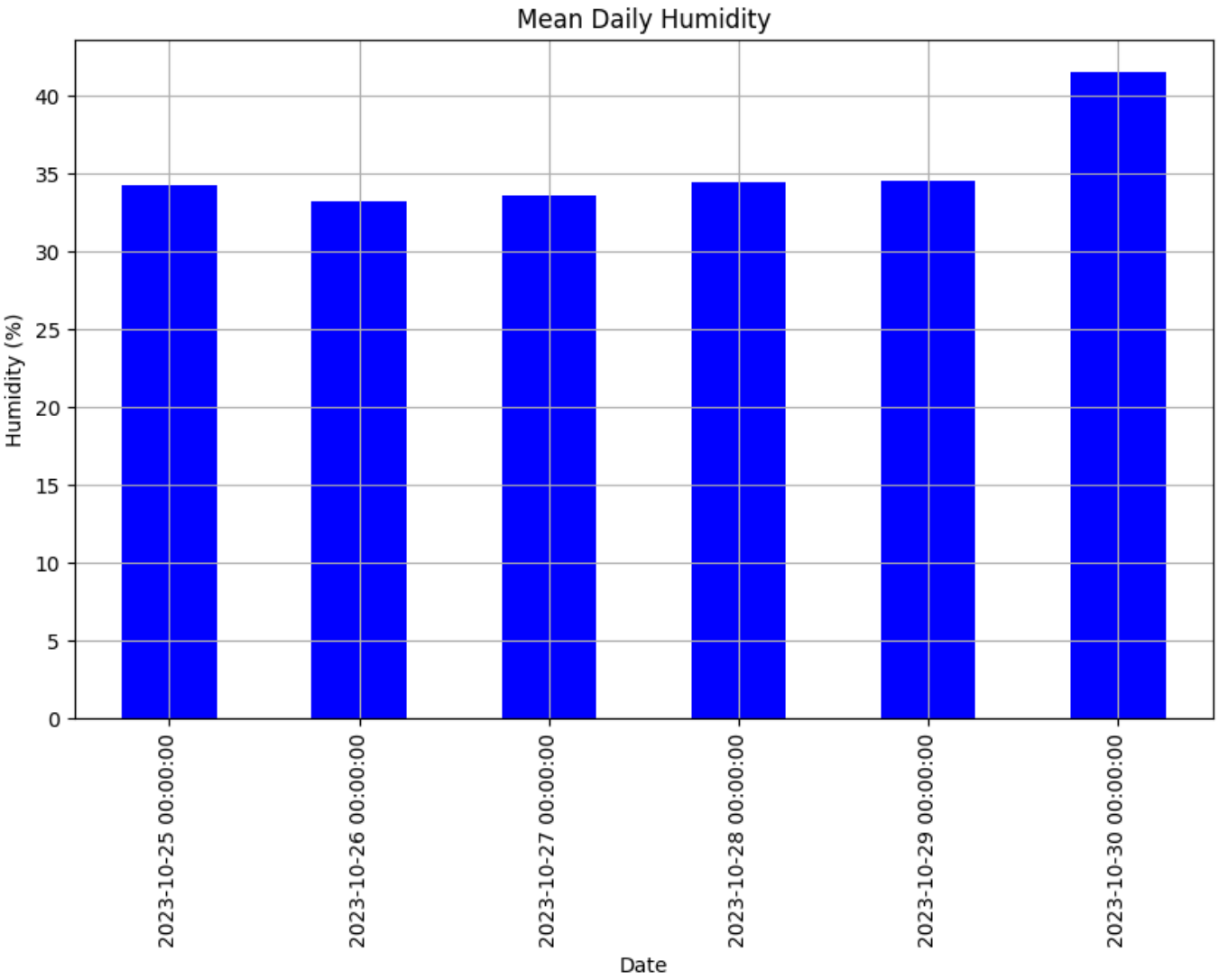
Timestamp	Temperature	humidity	wind_speed	weather_description
2023-10-25 06:00:00	29.99	30	3.15	scattered clouds
2023-10-25 09:00:00	30.67	28	3.55	scattered clouds
2023-10-25 12:00:00	30.23	27	5.39	scattered clouds
2023-10-25 15:00:00	26.19	31	4.05	clear sky
2023-10-25 18:00:00	23.68	40	3.66	clear sky
2023-10-25 21:00:00	21.44	49	1.62	few clouds
2023-10-26 00:00:00	20.01	55	0.29	few clouds
2023-10-26 03:00:00	24.58	40	1.43	scattered clouds
2023-10-26 06:00:00	30.17	23	4.54	scattered clouds
2023-10-26 09:00:00	32.12	18	5.11	clear sky
2023-10-26 12:00:00	29.53	23	5.13	few clouds
2023-10-26 15:00:00	25.40	28	3.91	broken clouds
2023-10-26 18:00:00	23.00	35	3.30	overcast clouds
2023-10-26 21:00:00	20.96	43	2.51	broken clouds
2023-10-27 00:00:00	19.22	49	1.40	broken clouds
2023-10-27 03:00:00	23.84	37	1.19	scattered clouds
2023-10-27 06:00:00	29.78	24	4.07	scattered clouds
2023-10-27 09:00:00	31.47	20	3.52	few clouds
2023-10-27 12:00:00	29.73	24	4.14	few clouds
2023-10-27 15:00:00	25.00	30	4.00	scattered clouds
2023-10-27 18:00:00	22.82	38	3.37	broken clouds
2023-10-27 21:00:00	20.88	46	2.51	scattered clouds
2023-10-28 00:00:00	19.34	51	1.55	scattered clouds
2023-10-28 03:00:00	23.97	39	1.71	clear sky
2023-10-28 06:00:00	29.53	26	3.38	clear sky
2023-10-28 09:00:00	31.25	21	2.25	clear sky
2023-10-28 12:00:00	29.82	27	1.25	clear sky
2023-10-28 15:00:00	25.50	29	3.19	clear sky
2023-10-28 18:00:00	22.93	37	3.46	scattered clouds
2023-10-28 21:00:00	20.62	45	0.47	broken clouds
2023-10-29 00:00:00	19.50	48	1.13	broken clouds
2023-10-29 03:00:00	24.43	36	1.03	scattered clouds
2023-10-29 06:00:00	29.71	27	3.59	scattered clouds
2023-10-29 09:00:00	31.48	22	1.53	few clouds
2023-10-29 12:00:00	30.15	29	1.03	few clouds
2023-10-29 15:00:00	25.65	32	1.04	clear sky
2023-10-29 18:00:00	23.04	38	2.08	clear sky
2023-10-29 21:00:00	21.01	44	0.45	clear sky
2023-10-30 00:00:00	20.03	47	1.56	clear sky
2023-10-30 03:00:00	24.83	36	1.70	clear sky

In [26]:
```
import matplotlib.pyplot as plt
daily_mean_temp = weather_df['Temperature'].resample('D').mean()
daily_mean_humidity = weather_df['humidity'].resample('D').mean()
daily_mean_wind_speed = weather_df['wind_speed'].resample('D').mean()
```

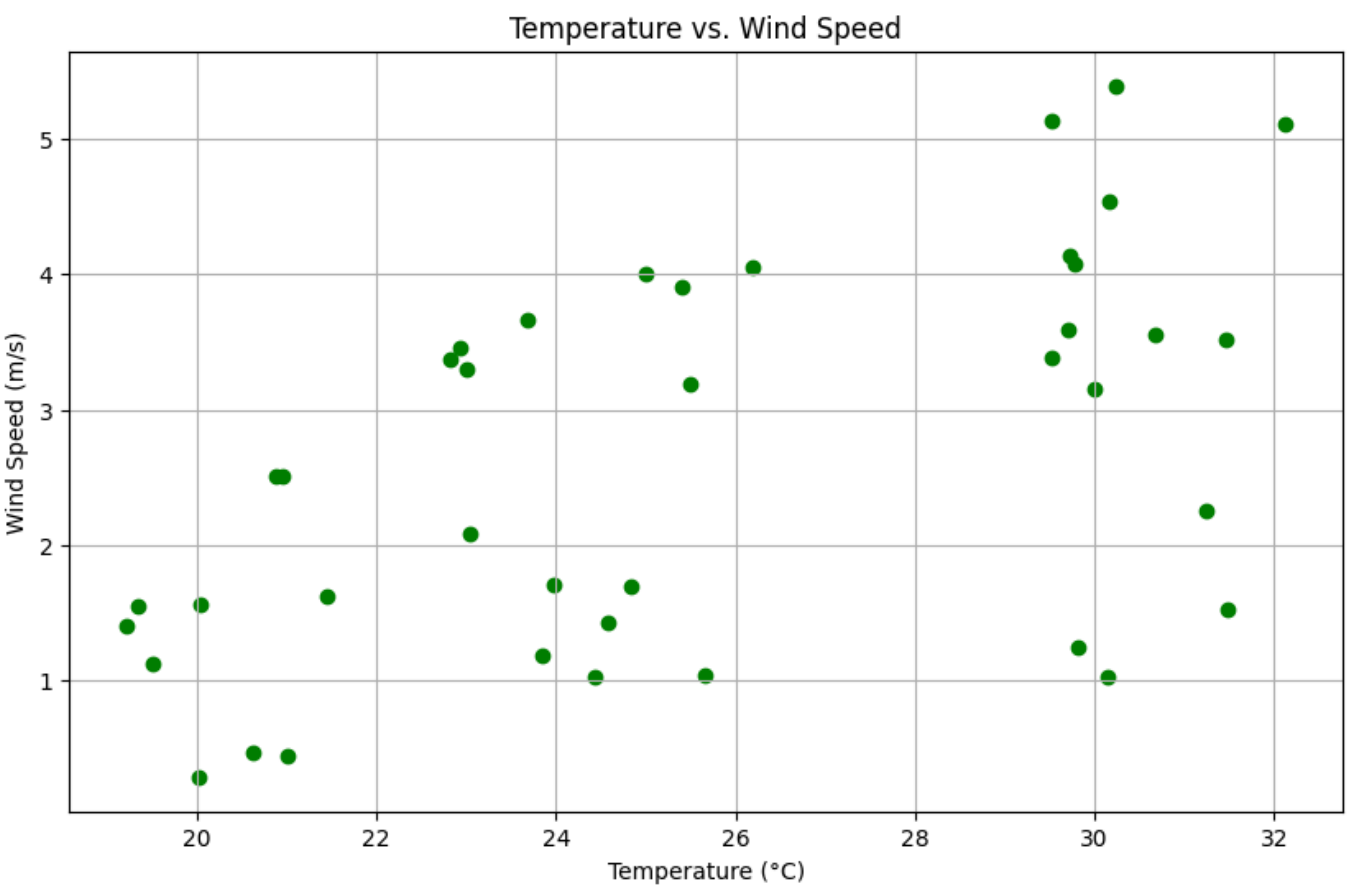
In [27]:
```
# Plot the mean daily temperature over time (Line plot)
plt.figure(figsize=(10, 6))
daily_mean_temp.plot(color='red', linestyle='-', marker='o')
plt.title('Mean Daily Temperature')
plt.xlabel('Date')
plt.ylabel('Temperature (°C)')
plt.grid(True)
plt.show()
```



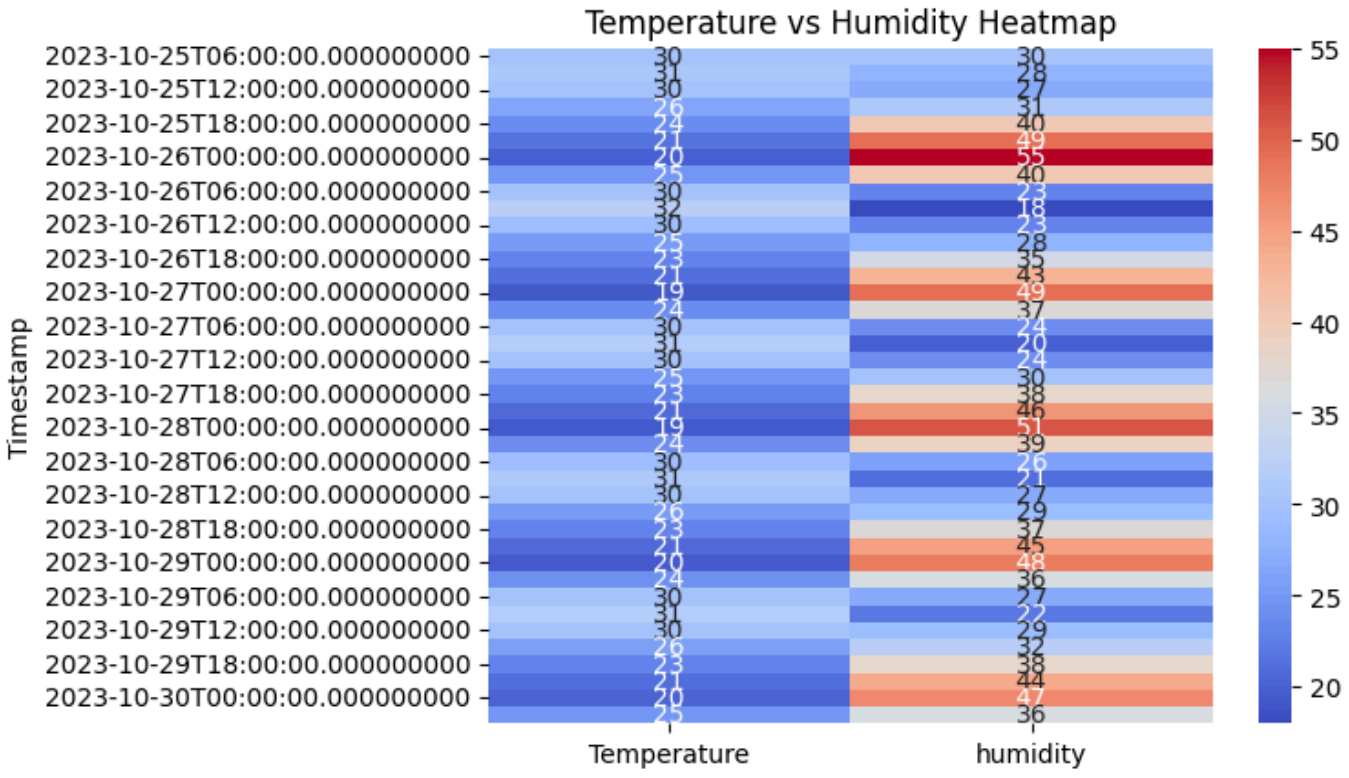
In [28]:
```
# Plot the mean daily humidity over time (Bar plot)
plt.figure(figsize=(10, 6))
daily_mean_humidity.plot(kind='bar', color='blue')
plt.title('Mean Daily Humidity')
plt.xlabel('Date')
plt.ylabel('Humidity (%)')
plt.grid(True)
plt.show()
```



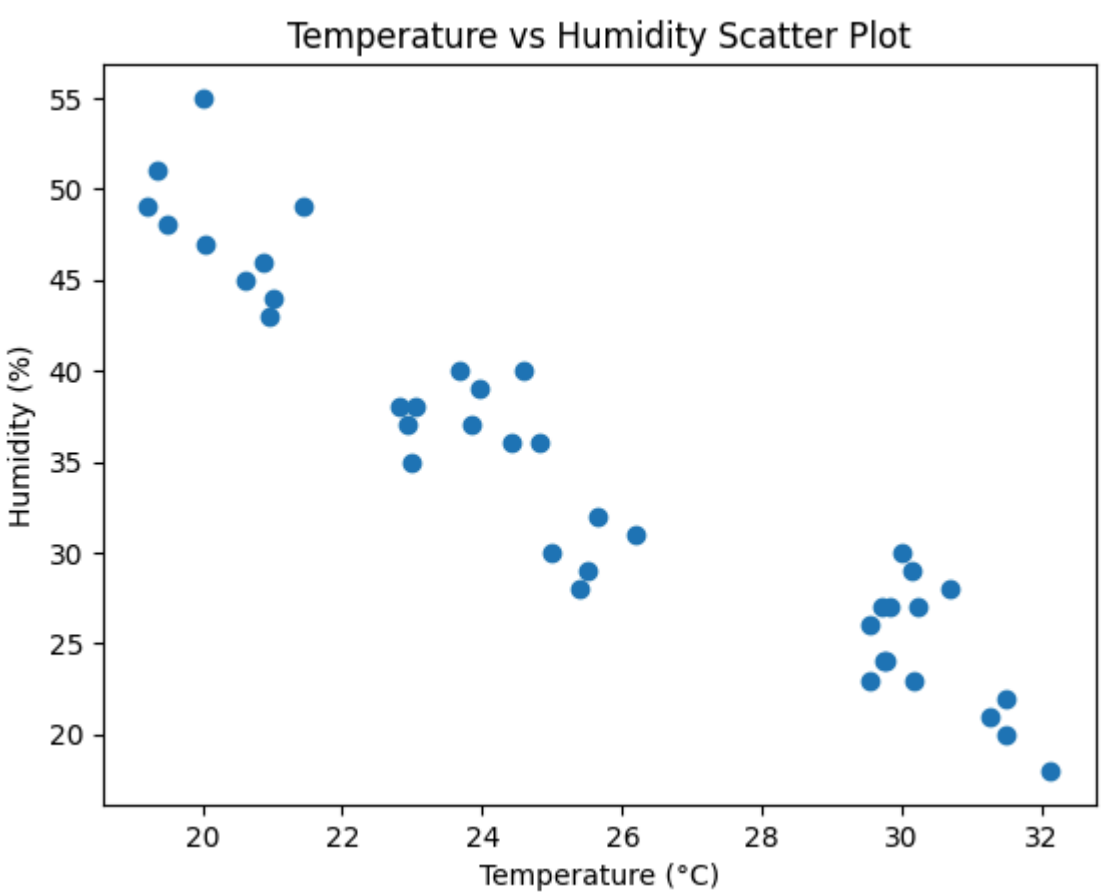
In [29]:
```
# Plot the relationship between temperature and wind speed (Scatter plot)
plt.figure(figsize=(10, 6))
plt.scatter(weather_df['Temperature'], weather_df['wind_speed'], color='green')
plt.title('Temperature vs. Wind Speed')
plt.xlabel('Temperature (°C)')
plt.ylabel('Wind Speed (m/s)')
plt.grid(True)
plt.show()
```



In [30]:
```
# Heatmap
import seaborn as sns
heatmap_data = weather_df[['Temperature', 'humidity']]
sns.heatmap(heatmap_data, annot=True, cmap='coolwarm')
plt.title('Temperature vs Humidity Heatmap')
plt.show()
```



In [31]:
```
# Create a scatter plot to visualize the relationship between temperature and humidity
plt.scatter(weather_df['Temperature'], weather_df['humidity'])
plt.xlabel('Temperature (°C)')
plt.ylabel('Humidity (%)')
plt.title('Temperature vs Humidity Scatter Plot')
plt.show()
```


Temperature vs Humidity Scatter Plot
Humidity (%)
Temperature (°C)
55
50
45
40
35
30
25
20
20
22
24
26
28
30
32

In [ ]: